

Machine learning methods for many body physics
Lectures for the *Advanced School on Quantum Science and Quantum
technology*

Giuseppe Carleo¹

September 8-9 2017, ICTP, Trieste, Italy

¹Institute for Theoretical Physics, ETH Zurich, Wolfgang-Pauli-Str. 27, 8093 Zurich, Switzerland

Chapter 1

An introduction to machine learning

Machine learning (ML) techniques are the modern answer to one of the most ancient desires of humanity: devising an Artificial Intelligence capable of independent thinking and problem solving. The applications of ML are ubiquitous, and you might have unconsciously made use of one of those. The tasks that can be solved are for example: language translation, face recognition, game playing.

How these things work then, and how can they be helpful for fundamental Sciences?

1.1 David Hilbert and his far-reaching problems

The starting point, is David Hilbert, and one of its 23 celebrated problems. In particular, the 13th problem is: consider a 7th degree equation:

$$x^7 + ax^3 + bx^2 + cx + 1 = 0,$$

can its solution $x(a, b, c)$ be expressed as the composition of a finite number of two-variable functions?

Answering this rather abstract mathematical problem led to the development of one of the most beautiful results in real analysis: the Kolmogorov-Arnold representation theorem. This theorem considers a generic continuous function of n variables $F(x_1, x_2, \dots, x_n)$, and demonstrates that it can be approximated using only a finite number of one-variable functions. One of the most recent refinements, due to Sprechner [1], in particular states that:

$$F(x_1, x_2, \dots, x_n) = \sum_{q=0}^{2n} \Phi \left(\sum_{p=1}^n \lambda_p \phi(x_p + \eta q) + q \right), \quad (1.1)$$

where Φ and ϕ are one-dimensional continuous functions. In particular $0 \leq \phi(y) \leq 1$, with $0 \leq y \leq 1$, and η, λ_q are parameters to be determined.

This theorem is particularly important because, in a sense, demystifies the complexity of high-dimensional functions: in order to express a complicated high-dimensional function you “just” need to specify two one-dimensional functions. The composition of those functions then gives the full function.

1.2 Artificial Neural networks: the machine

What has this theorem to do with Facebook, Google, or self-driving cars then? In most of those applications, we have a task to be solved whose solution can be expressed as a complex high-dimensional function. For example, let us take the case of identifying faces in a picture. To simplify, let us assume that the picture is monochromatic, and it can be seen, when digitalized, as a set $\mathbf{x} = b_1, b_2 \dots b_n$ of bits $b_i = 0, 1$ (signaling if the given pixel should be black or white). Then, we can ask a machine to give the probability that in a given picture a certain person exists: this can be formulated as: find a function $F_{\text{image}}(\mathbf{x}) = P_{\text{inthere}}(\mathbf{x})$. Analogously, other tasks can be written in this form, like in the case of game playing, where f could be for example the game policy.

The field of ML is mainly concerned then with the problem of finding compact approximations of highly-dimensional functions.

In this respect, Kolmogorov-Arnold theorem is particular important because at least gives us a hope that representing those functions, no-matter how complicated, requires “only” the need to approximate possibly easier one-dimensional functions. This goal is achieved through several *learning* paradigms, i.e. ways of using or generating relevant information to find the best functions.

Historically, the form (1.1) is not the one that typically is adopted in applications. Instead, one typically relies on the notion of artificial neural network. An artificial neural network is nothing but a highly-dimensional function, composition of simple one-dimensional functions, and depending on internal parameters. Taking inspiration from the biology of the brain, the specific functions are taken to be for example of the logistic/sigmoidal form:

$$\phi(x) = \frac{1}{1 + \exp(-x)}.$$

In a feed-forward neural network we would take for example a linear combination of the input, to define a new set of variables (which defines the first layer of the network):

$$y_j = \phi \left(\sum_i w_{ij} x_i + b_j \right),$$

where the matrix w_{ij} and the *bias* vector b_j are arbitrary parameters. Then, we can *feed* those variables into another layer:

$$z_k = \phi \left(\sum_j w'_{jk} y_j + b'_k \right),$$

and so on, until we reach the final layer, where for example we take a single output variable:

$$\begin{aligned} o &= \phi \left(\sum_k w''_k z_k + b'' \right) \\ &\equiv f(\mathbf{x}), \end{aligned}$$

in the sense that the full function can be seen as a nested composition of non-linear vector functions:

$$F(\mathbf{x}; w, b) = \phi(A(B(C(\mathbf{x})))).$$

1.3 Supervised Learning

With the machine architecture specified by a set of parameters $\mathbf{p} = p_1 \dots p_{N_p}$ (for example the weights and bias in the previous equations), the remaining task is to find a convenient strategy to numerically determine the weights and the structure of the network. The conceptually simplest approach is the so-called supervised learning.

In this context we assume that a collection of large, pre-existing data $\mathbf{x}_1, \mathbf{x}_2, \dots \mathbf{x}_{N_s}$ exists, such that it is known in advance what the desired output of the machine should be on those, i.e. we also know the associated *labels*: $y_i \equiv F_{\text{ideal}}(\mathbf{x}_i)$. Goal of the supervised learning is to find the best neural-network weights such as $F(\mathbf{x}; \mathbf{p}) \simeq F_{\text{ideal}}(\mathbf{x})$, where the ideal function is known only at the points in the dataset.

In order to quantify the quality of the machine at doing his task, we can define the loss function:

$$\mathcal{L}(\mathbf{p}) = \frac{1}{N_s} \sum_i^{N_s} |F(\mathbf{x}_i; \mathbf{p}) - y_i|, \quad (1.2)$$

which attains a minimum value of zero for perfect reconstruction. The supervised learning problem is therefore a conceptually simple inference problem or, alternatively, a non-linear fitting problem.

1.3.1 Stochastic gradient descent

With a loss function specified (relevant possibilities include, for example, the choice of the L_1 or L_2 norm, in expression (1.2)), the numerical goal is to minimize it. The most widely approach used in the community is the stochastic gradient descent. This method is a relative of the standard gradient descent algorithm, i.e. an iterative approach in which the parameters are update according to:

$$\mathbf{p}^{(s+1)} = \mathbf{p}^{(s)} - \eta G(\mathbf{p}^{(s)}), \quad (1.3)$$

where $G(\mathbf{p}^{(k)})$ is some *approximation* of the gradient of the function to be optimized, and η is a (small) parameter known as the learning rate. The crucial difference between the standard GD and the SGD is in how the parameter updates are computed.

In the standard GD, the G is just the standard gradient, i.e.:

$$\begin{aligned} G_{\text{GD}}(p_1 \dots p_{N_p}) &= \nabla_p \mathcal{L}(\mathbf{p}), \\ &= \frac{1}{N_s} \sum_i^{N_s} \nabla_p |F(\mathbf{x}_i; W) - y_i| \end{aligned}$$

in the SGD case, instead, one consider a *batch approximation*:

$$G_{\text{SGD}}(p_1 \dots p_{N_p}) = \frac{1}{N_b} \sum_i^{N_b} \nabla_p |F(\mathbf{x}_i; \mathbf{p}) - y_i|,$$

where in the last expression the sum runs on a subset consisting of $N_b \ll N_s$ data points. In the limit of large N_s , the true gradient can be interpreted as an expectation value over the (unknown) probability distribution according to which the data $\mathbf{x}$ is distributed: $\Pi(\mathbf{x})$, such that

$$G_{\text{GD}}(p_1 \dots p_{N_p}) = \langle \langle \nabla_p |F(\mathbf{x}; \mathbf{p}) - y| \rangle \rangle_{\Pi}.$$

If the data belonging to the batch is randomly drawn (with replacement) from the original dataset, then $G_{\text{SGD}}(\mathbf{p})$ can be seen as a noisy approximation of the true gradient. Let us assume for simplicity that all the components of the gradient are subjected to the same amount of gaussian noise with a variance $\sigma \propto \frac{1}{\sqrt{N_b}}$, i.e.

$$G_k^{\text{SGD}}(p_1 \dots p_{N_p}) = \nabla_p \mathcal{L}(\mathbf{p}) + \text{Normal}(\sigma). \quad (1.4)$$

We can then compare Eq. (1.3) to the discretized Langevin equation:

$$p_k^{s+1} = p_k^s - \delta_t \partial_{p_k} \mathcal{L}(W) + \text{Normal}(\sqrt{2\delta_t T}), \quad (1.5)$$

where δ_t is a small time step. This equation samples the Boltzmann distribution

$$\Pi_B(p_1 \dots p_M) = e^{-\frac{\mathcal{L}(\mathbf{p})}{T}}, \quad (1.6)$$

which in the limit $T \rightarrow 0$ would converge to the minimum value (ground state) of the effective classical energy $\mathcal{L}(\mathbf{p})$, i.e. to $\min_{\mathbf{p}} \mathcal{L}(\mathbf{p})$. We therefore see that the variance of the gradient corresponds to the effective temperature as

$$\sigma^2 = \text{var}(\partial_{p_k} \mathcal{L}) = 2T/\delta_t \quad (1.7)$$

$$\eta = \delta_t. \quad (1.8)$$

Since we want to find the variational ground state, we should have a scheme in which the temperature is gradually decreased at each optimization step, i.e. $T_1 > T_2 > T_3 \dots$, as in the simulated annealing optimization protocol. The first thing we notice is that $\sigma^2 \simeq \frac{1}{N_s}$, decreases like the number of samples in the batch, therefore

$$T = \frac{\eta \text{var}(\partial_{p_k} \mathcal{L})}{2} \quad (1.9)$$

$$\propto \frac{\eta}{N_s} \text{var}(\partial_{p_k} \mathcal{L}) \quad (1.10)$$

A convenient way to reduce the temperature is then reduce the learning rate with the step number, for example a common choice is $\eta(s) = \eta_0/\sqrt{s+1}$ or, alternatively (but much less used), to increase the number of samples in the batch with the iteration count.

1.3.2 Example 1: MNIST database

1.3.3 Example 2: Classifying phases of matter

Chapter 2

Unsupervised Learning

It is often the case that in the most interesting applications the label is not given at training time. For example, consider the case of classifying phases of matter: it might be the case that we do not know beforehand what phases we should expect. In this case, a more involved (and powerful) form of machine learning exists, dubbed *unsupervised learning*. In this context, there are several goals that can be achieved in the presence of an unlabeled set of data. Here we consider the case in which we assume that the data $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{N_s}$ is distributed according to some unknown probability distribution $\Pi(\mathbf{x})$ that we want to determine. The goal of our artificial neural network is then in this case to approximate this high-dimensional function, i.e. we should find the best weights such that: $\frac{F(\mathbf{x}; \mathbf{p})}{Z(\mathbf{p})} \simeq \Pi(\mathbf{x})$, where we have assumed that we impose $F(\mathbf{x}) \geq 0$, and define the normalization $Z(\mathbf{p}) = \sum_{\{\mathbf{x}\}} F(\mathbf{x}; \mathbf{p})$, such that $\frac{F(\mathbf{x}; \mathbf{p})}{Z(\mathbf{p})}$ is a properly defined probability density. First of all, we need to specify what we mean by the *approximately equal* sign in the previous equation. To do so, we need some sensible measure of the distance between two probability distributions.

2.1 Kullback–Leibler divergence

A particularly successful (and practically useful) measure of the difference between two probability distributions is given by the Kullback-Leibler (KL) divergence. Taking the case of discrete-valued probabilities, it is defined as:

$$D_{\text{KL}}(\Pi \parallel F) = \sum_{\{\mathbf{x}\}} \Pi(\mathbf{x}) \log \frac{\Pi(\mathbf{x})}{F(\mathbf{x}; \mathbf{p})/Z(\mathbf{p})}. \quad (2.1)$$

This is not, strictly speaking, a metrics in the space of probabilities (notice that is not symmetric in the exchange of Π and F , for example) however it attains a minimum when the two probabilities are equal. From an information theory perspective, a non-zero KL divergence signals how large is the information loss when coding the given data with the approximate probability distribution. Minimizing this quantity is therefore certainly a meaningful task.

From a more practical point of view, the great advantage of the KL divergence is

that its gradient can be expressed exactly as the difference of two expectation values:

$$\partial_{p_k} D_{\text{KL}}(\Pi \parallel F) = \sum_{\{\mathbf{x}\}} \Pi(\mathbf{x}) (-\partial_{p_k} \log F(\mathbf{x}; \mathbf{p}) + \partial_{p_k} \log Z(\mathbf{p})) \quad (2.2)$$

$$= -\langle \langle \partial_{p_k} \log F(\mathbf{x}; \mathbf{p}) \rangle \rangle_{\Pi} + \frac{\sum_{\{\mathbf{x}\}} \partial_{p_k} \log F(\mathbf{x}; \mathbf{p})}{Z(\mathbf{p})} \quad (2.3)$$

$$= -\langle \langle \partial_{p_k} \log F(\mathbf{x}; \mathbf{p}) \rangle \rangle_{\Pi} + \langle \langle \partial_{p_k} \log F(\mathbf{x}; \mathbf{p}) \rangle \rangle_F. \quad (2.4)$$

In particular, expectation values over Π can be easily computed using the available data, and without knowing explicitly the value of Π :

$$\langle \langle \partial_{p_k} \log F(\mathbf{x}; \mathbf{p}) \rangle \rangle \simeq \frac{1}{N_s} \sum_i \partial_{p_k} \log F(\mathbf{x}_i; \mathbf{p}),$$

whereas the second expectation value can be obtained devising a procedure that generate samples from our neural network.

2.2 Restricted Boltzmann Machines

In this section we introduce in more detail a specific class of models for machine learning, especially useful for unsupervised learning and, as we will show, for quantum mechanics. This class has strong ties to Statistical Physics, and, in some sense, it is one of the most natural ways to approach the field coming from Physics.

In this case, we consider as a state vector $\mathbf{x} = \sigma_1, \sigma_2 \dots \sigma_N$, i.e. the collection of N spin 1/2 variables, each of which taking the value $\sigma_i = \pm 1$. This choice is completely equivalent to taking a set of binary variables, but has a more transparent physical meaning. We then introduce a network called Restricted Boltzmann Machine (RBM), which is defined as:

$$F_{\text{rbm}}(\sigma_1, \sigma_2, \dots \sigma_N) = \sum_{\{h\}} \exp \left[\sum_{ij} W_{ij} \sigma_i h_j + \sum_j h_j b_j + \sum_i \sigma_i a_i \right], \quad (2.5)$$

where the network parameters are W, a , and b . This architecture corresponds to the partition function of a gas of M hidden units ($h_j = \pm 1$) connected to the physical spins (σ_i^z). Since the connections are allowed only between hidden and visible units, but not between hidden units, nor between visible units, this architecture is called *restricted*. Because of this restriction, however it is easy to compute F explicitly. Indeed

$$\sum_{\{h\}} \exp \left[\sum_{ij} W_{ij} \sigma_i h_j + \sum_j h_j b_j + \sum_i \sigma_i a_i \right] = \quad (2.6)$$

$$e^{\sum_i \sigma_i a_i} \times \sum_{\{h\}} \Pi_j \exp \left[\sum_i W_{ij} \sigma_i h_j + h_j b_j \right] = \quad (2.7)$$

$$e^{\sum_i \sigma_i a_i} \times \Pi_j \left(\exp \left[\sum_i W_{ij} \sigma_i + b_j \right] + \exp \left[-\sum_i W_{ij} \sigma_i - b_j \right] \right) = \quad (2.8)$$

$$e^{\sum_i \sigma_i a_i} \times \Pi_j 2 \cosh \left[\sum_i W_{ij} \sigma_i + b_j \right]. \quad (2.9)$$

2.3 Gibbs Sampling

An attractive feature of those models (apart from the nice physical interpretation) is that they allow for an efficient sampling strategy. In this context, Gibbs sampling is often adopted. Gibbs sampling can be seen as a special case of the more widely known Metropolis-Hastings algorithm (see Appendix A). Let us introduce the quantity

$$P(\sigma, h) = \exp \left[\sum_{ij} W_{ij} \sigma_i h_j + \sum_j h_j b_j + \sum_i \sigma_i a_i \right], \quad (2.10)$$

which is the joint probability density (apart from a global normalization) of the physical and hidden units. [3]

The idea of alternate Gibbs sampling is then to devise a two step Markov-chain sampling with transition probabilities:

$$T_\sigma((\sigma, h) \rightarrow (\sigma', h)) = \frac{P(\sigma', h)}{\sum_{\sigma''} P(\sigma'', h)} = P(\sigma' | h) \quad (2.11)$$

$$T_h((\sigma, h) \rightarrow (\sigma, h')) = \frac{P(\sigma, h')}{\sum_{h''} P(\sigma, h'')} = P(h' | \sigma). \quad (2.12)$$

The acceptance probability for these two type of moves can be readily computed using the Metropolis-Hasting acceptance rule:

$$A(\mathbf{x} \rightarrow \mathbf{x}') = \min \left(1, \frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x})}{T(\mathbf{x} \rightarrow \mathbf{x}')} \right), \quad (2.13)$$

where in one case spin configuration are changed, $\mathbf{x}' = (\sigma', h)$ and in the other case hidden variable configuration are changed, thus $\mathbf{x}' = (\sigma, h')$. For example, in the first case the acceptance probability reads:

$$\begin{aligned} A((\sigma, h) \rightarrow (\sigma', h)) &= \min \left\{ 1, \frac{P(\sigma', h)}{P(\sigma, h)} \times \frac{T_\sigma((\sigma', h) \rightarrow (\sigma, h))}{T_\sigma((\sigma, h) \rightarrow (\sigma', h))} \right\} \\ &= \min \left\{ 1, \frac{P(\sigma', h)}{P(\sigma, h)} \times \frac{P(\sigma | h)}{P(\sigma' | h)} \right\} \\ &= 1, \end{aligned} \quad (2.14)$$

where in the last line we have used the fact that

$$\frac{P(\sigma | h)}{P(\sigma' | h)} = \frac{\frac{P(\sigma, h)}{\sum_{\sigma''} P(\sigma'', h)}}{\frac{P(\sigma', h)}{\sum_{\sigma''} P(\sigma'', h)}} = \frac{P(\sigma, h)}{P(\sigma', h)}. \quad (2.15)$$

The same reasoning can be done for moves that change the hidden units only, and one gets an acceptance of 1 as well. The important point is that $P(\sigma | h)$ and $P(h | \sigma)$ can be computed exactly for an RBM.

For example, we have that:

$$\begin{aligned} P(h | \sigma) &= \frac{P(\sigma, h)}{\sum_{h''} P(\sigma, h'')} \\ &= \frac{e^{\sum_i \sigma_i a_i} \times \prod_j \exp [\sum_i W_{ij} \sigma_i h_j + b_j h_j]}{e^{\sum_i \sigma_i a_i} \times \prod_j 2 \cosh [\sum_i W_{ij} \sigma_i + b_j]}, \end{aligned} \quad (2.16)$$

and each hidden variable has a probability which is independent on the value of the other hidden variables. We have:

$$P(h_j = 1|\sigma) = \text{Logistic}(2\theta_j) \quad (2.17)$$

$$P(h_j = -1|\sigma) = \text{Logistic}(-2\theta_j), \quad (2.18)$$

where $\text{Logistic}(x) = \frac{1}{1+\exp(-x)}$ and $\theta_j = \sum_i W_{ij}\sigma_i + b_j$. A similar expression can be derived also for the other conditional probability, which reads:

$$\begin{aligned} P(h|\sigma) &= \frac{P(\sigma, h)}{P(h)} \\ &= \frac{e^{\sum_j h_j b_j} \times \prod_i \exp \left[\sum_j W_{ij} \sigma_i h_j + a_i \sigma_i \right]}{e^{\sum_j h_j b_j} \times \prod_i 2 \cosh \left[\sum_j W_{ij} h_j + a_i \right]}, \end{aligned} \quad (2.19)$$

and each spin variable has a probability which is independent on the value of the other spin variables. We then have:

$$P(\sigma_i = 1|h) = \text{Logistic}(2\gamma_i) \quad (2.20)$$

$$P(\sigma_i = -1|h) = \text{Logistic}(-2\gamma_i), \quad (2.21)$$

where $\gamma_i = \sum_j W_{ij}h_j^z + a_i$. Proposing spin and hidden variable configurations according to the Gibbs transition probability is therefore very easy and consists in the following:

1. Generate N random numbers $r_i \in [0, 1)$.
2. Set the i -th spin with probability $P(\sigma_i = 1|h) = \text{Logistic}(2\gamma_i)$, i.e. if $P(\sigma_i = 1|h) > r_i$ then set $\sigma'_i = 1$ otherwise $\sigma'_i = -1$.
3. Generate M random number $l_j \in [0, 1)$.
4. Set the j -th hidden unit with probability $P(h_j = 1|\sigma) = \text{Logistic}(2\theta_j)$, i.e. if $P(h_j = 1|\sigma) > l_j$ then set $h'_j = 1$ otherwise $h'_j = -1$.

Repeating these steps N_s times, we then generate spin configurations which are sampled from $F_{\text{rbm}}(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z)$ –Eq. (2.5). Notice that this scheme is rather easy to implement since we do not need to perform a Metropolis-Hastings test at each step of the Markov chain, given that all moves are accepted (see Appendix A for details).

2.4 Unsupervised learning with RBMs

One of the most natural applications for RBMs is for unsupervised learning. In this case we immediately use the expression (2.2) to minimize the KL divergence, and learn the parameters of the RBM. In particular, define

$$D_k(\mathbf{x}; \mathbf{p}) = \partial_{p_k} \log F_{\text{rbm}}(\mathbf{x}; \mathbf{p}), \quad (2.22)$$

which in the case of the RBM simply read:

$$D_{a_i}(\sigma) = \frac{1}{2}\sigma_i, \quad (2.23)$$

$$D_{b_j}(\sigma) = \frac{1}{2}\tanh(\theta_j), \quad (2.24)$$

$$D_{W_{ij}}(\sigma) = \frac{1}{2}\tanh(\theta_j)\sigma_i. \quad (2.25)$$

then the gradient of the KL reads:

$$\partial_{p_k} D_{\text{KL}}(\Pi \parallel F) = -\langle\langle D_k(\mathbf{x}; \mathbf{p}) \rangle\rangle_{\Pi} + \langle\langle D_k(\mathbf{x}; \mathbf{p}) \rangle\rangle_{F_{\text{rbm}}}. \quad (2.26)$$

A particularly popular strategy of computing this gradient is the so-called *contrastive divergence* method of Hinton [4].

In this case one usually takes a very small batch size (i.e. a small value of N_s in the Gibbs sampling procedure, often as small as $N_s = 1$). This implies a very large statistical error on the gradient, which however is strongly mitigated by the fact that the starting configuration for the Gibbs sampling is not chosen at random, but is one of the elements in the dataset. This means that errors between the two terms in the gradient tend to cancel out, because of correlated sampling.

2.4.1 Example 3: Forging hand-writing

2.4.2 Example 4: Analytical learning of classical many-body models

As an example application in many-body physics, we can ask the following question: imagine that we have observed several thermodynamic samples, generated according to some Boltzmann weight:

$$P(\sigma_1, \dots, \sigma_N) = \frac{e^{-\beta E(\sigma_1, \dots, \sigma_N)}}{Z}, \quad (2.27)$$

and we want to determine the form of the (a-priori unknown) energy function $E(\sigma_1 \dots \sigma_N)$. This is feasible through unsupervised learning. The question one might ask is then: what kind of classical models can I efficiently represent with an RBM network?

The answer to this question is very simple: all physical models can be represented efficiently, i.e. with a number of hidden units which grows only polynomially with system size. Consider for example the case of a generic two-body interaction:

$$E(\sigma_1 \dots \sigma_N) = \sum_{i < j} V_{ij} \sigma_i \sigma_j, \quad (2.28)$$

then in this case it is easy to show that the interaction between the two spins can be mediated, in the RBM, introducing a hidden unit $h_{V_{ij}}$ which takes care of each bond. This means that we just need to solve:

$$e^{-\beta V_{ij} \sigma_i \sigma_j} = \sum_{\{h_V\}} e^{\sigma_i W_{iV} h_V + \sigma_j W_{jV} h_V}, \quad (2.29)$$

or

$$e^{-\beta V_{ij}} = 2 \cosh(W_{iV} + W_{jV}) \quad (2.30)$$

$$e^{\beta V_{ij}} = 2 \cosh(W_{iV} - W_{jV}), \quad (2.31)$$

to determine the weights W_{iV} and W_{jV} . In general then, any few-body interaction can be expressed in the RBM form exactly, provided that dedicated hidden units are associated to each of the local interactions, as exemplified for the two-body case.

2.4.3 Example 5: Equivalent representations of the Ising model

Torlai and Melko, Phys. Rev. B 94, 165134 (2016)

2.4.4 Example 6: Forging quantum measurements

Quantum state tomography is an example where those unsupervised-learning techniques can be more or less immediately adopted. In Ref. [6] it has been shown for example how to use RBMs to reconstruct the state of a quantum system.

Chapter 3

Quantum Mechanics as a Machine Learning Problem

We have seen in the previous lectures that every machine learning approach has two fundamental ingredients.

1. *The machine*: typically an artificial neural-network, it is a highly dimensional (non-linear) function $F(\mathbf{x}; p_1 \dots p_{N_p})$ of the parameters $p_1 \dots p_{N_p}$
2. *The learning*: the parameters $\mathbf{p}$ are learned on the basis of a stochastic optimization, that minimizes some average loss function $\langle \mathcal{L} \rangle(\mathbf{p})$ on a dataset $\mathbf{x}_1, \mathbf{x}_2 \dots \mathbf{x}_{N_s}$. For example $\mathcal{L}(\mathbf{x}_i) = |F(\mathbf{x}_i; \mathbf{p}) - \mathbf{y}_i|$, in the supervised learning setting with expected labels $\mathbf{y}_i$.

On the other hand, the central goal in quantum mechanics is to find a solution to Schroedinger equation

$$H|\Psi_i\rangle = E_i|\Psi_i\rangle, \quad (3.1)$$

for $i = 0, 1, \dots$ and $E_0 < E_1 < \dots$. How can we reduce quantum mechanics then to a machine learning problem?

First of all, I will address the requirement 2, which has been done by pioneers in computational quantum physics like Bill McMillan, in the 60s. Then, I will address requirement 1, which has been done instead only very recently, thus completing the connection between machine learning and quantum mechanics.

3.1 Variational Monte Carlo

To satisfy requirement 2, we need to transform this eigenvalue problem (3.1) into a stochastic optimization problem. To achieve this, we start from an alternative formulation of Schroedinger's equation, based on the variational principle. In particular, consider the energy functional:

$$E[\Psi] = \langle \Psi | H | \Psi \rangle \geq E_0, \quad (3.2)$$

where Ψ is some arbitrary physical state, and E_0 is the exact ground-state energy of the Hamiltonian H . From the variational theorem it is then clear that one can find the exact ground-state wave-function as the solution of the optimization problem:

$$\Psi_0 = \operatorname{argmin}_{\Psi} E[\Psi]. \quad (3.3)$$

For an arbitrary state Ψ , however it is seldom possible to compute analytically the energy functional, since it involves integrals over a high-dimensional space. To solve this problem, in the 60s McMillan realized that the energy functional can be computed *stochastically*. [2] In particular, the Variational Monte Carlo method is rooted into the observation that expectation values like (3.2) can be written as statistical averages over a suitable probability distribution.

Let us assume that our Hilbert space is spanned by the many-body kets $|\mathbf{x}\rangle$. These in practice depend on the system in exam. For example in the case of spins 1/2 we would typically have $|\mathbf{x}\rangle = |\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z\rangle$, for second-quantized fermions $|\mathbf{x}\rangle = |n_1, n_2, \dots, n_N\rangle$, for particles in continuous space $|\mathbf{x}\rangle = |\vec{r}_1, \vec{r}_2, \dots, \vec{r}_N\rangle$. The only difference is of course that in the first two cases one has a discrete set of quantum numbers, whereas in the latter case the degrees of freedom are continuous. In both cases we will denote sums over the Hilbert space with discrete sums, although one should always bear in mind that in the case of continuous variables these sums must be interpreted as integrals. In particular we will use the closure relation $\sum_{\mathbf{x}} |\mathbf{x}\rangle \langle \mathbf{x}| = 1$.

3.1.1 Stochastic Estimates of Properties

Using the closure relation, we can rewrite a generic quantum expectation value of some operator O as

$$\frac{\langle \Psi | O | \Psi \rangle}{\langle \Psi | \Psi \rangle} = \frac{\sum_{\mathbf{x}, \mathbf{x}'} \langle \Psi | \mathbf{x} \rangle \langle \mathbf{x} | O | \mathbf{x}' \rangle \langle \mathbf{x}' | \Psi \rangle}{\sum_{\mathbf{x}} \langle \Psi | \mathbf{x} \rangle \langle \mathbf{x} | \Psi \rangle} \quad (3.4)$$

$$= \frac{\sum_{\mathbf{x}, \mathbf{x}'} \Psi^*(\mathbf{x}) O_{\mathbf{x}\mathbf{x}'} \Psi(\mathbf{x}')}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2}. \quad (3.5)$$

There can be, in general, two cases:

1. The operator O is diagonal in the computational basis, i.e. $O_{\mathbf{x}\mathbf{x}'} = \delta_{\mathbf{x}\mathbf{x}'} O(\mathbf{x})$. Then

$$\frac{\langle \Psi | O | \Psi \rangle}{\langle \Psi | \Psi \rangle} = \frac{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2 O(\mathbf{x})}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \quad (3.6)$$

$$\equiv \langle \langle O \rangle \rangle, \quad (3.7)$$

where $\langle \langle \dots \rangle \rangle$ denote *statistical* expectation values over the probability distribution $\Pi(\mathbf{x}) = |\Psi(\mathbf{x})|^2$. In other words, in this case quantum expectation values are completely equivalent to averaging over Hilbert-space states sampled according to the square-modulus of the wave-function.

2. The operator O is off-diagonal in the computational basis. Then, we can define an auxiliary diagonal operator (often called, in a somehow misleading fashion, *local* operator or estimator)

$$O_{\text{loc}}(\mathbf{x}) = \sum_{\mathbf{x}'} O_{\mathbf{x}\mathbf{x}'} \frac{\Psi(\mathbf{x}')}{\Psi(\mathbf{x})}, \quad (3.8)$$

such that it is easily proven that

$$\frac{\langle \Psi | O | \Psi \rangle}{\langle \Psi | \Psi \rangle} = \frac{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2 O_{\text{loc}}(\mathbf{x})}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \quad (3.9)$$

$$\equiv \langle \langle O_{\text{loc}} \rangle \rangle. \quad (3.10)$$

For any observable, then, we can always compute expectation values over arbitrary wave-functions as statistical averages. In the case of off-diagonal operators, it should be noticed that the sum $\sum_{\mathbf{x}'} O_{\mathbf{x}\mathbf{x}'} \frac{\Psi(\mathbf{x}')}{\Psi(\mathbf{x})}$, is extended to the tiny portion of the Hilbert space for which $\mathbf{x}'$ is such that $O_{\mathbf{x}\mathbf{x}'} \neq 0$. For the great majority of physical observables, and for a given $\mathbf{x}$, the number of elements $\mathbf{x}'$ connected by those matrix elements is polynomial in the system size, thus the summation can be carried systematically. This has to be contrasted instead to the summations in $\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2$, where one typically has an exponentially large number of possible values of $\mathbf{x}$ on which to perform the summation, and therefore cannot be done by brute-force. The powerful idea of the Variational Monte Carlo, is therefore to replace these sums over exponentially many states, with a statistical average over a large but finite set of states sampled according to the probability distribution $\Pi(\mathbf{x})$. We therefore have a way to compute, stochastically, the expectation value of *all* the properties of interest. For example we might want to compute the expectation value of σ_i^x for a spin system, the expectation value of $c_i^\dagger c_j$ for fermions, or even the expectation value of the interaction energy $W_{ee}(\vec{r}_1 \dots \vec{r}_N)$ for our electronic structure problems.

3.1.1.1 Energy

An immediate corollary of the previously presented scheme, is that also the expectation value of the Hamiltonian H (which is itself a generic off-diagonal operator) can be computed using the estimator (3.10). Historically, the local estimator associated to the Hamiltonian is called “local energy”:

$$E_{\text{loc}}(\mathbf{x}) = \sum_{\mathbf{x}'} H_{\mathbf{x}\mathbf{x}'} \frac{\Psi(\mathbf{x}')}{\Psi(\mathbf{x})}. \quad (3.11)$$

3.2 Stochastic Variational Optimization

The final goal we want to achieve here is to optimize the variational energy. In practice, we assume that the wave function depends on some (possibly millions of) parameters $\mathbf{p} = p_1, \dots, p_M$. We have seen that the expectation value of the energy can be written as a statistical average of the form

$$\langle H \rangle \simeq \langle \langle E_{\text{loc}} \rangle \rangle. \quad (3.12)$$

It is easy to show that also the gradient of the energy can be written under to form of the expectation value of some stochastic variable. In particular, define

$$D_k(\mathbf{x}) = \frac{\partial_{p_k} \Psi(\mathbf{x})}{\Psi(\mathbf{x})}, \quad (3.13)$$

then

$$\begin{aligned} \partial_{p_k} \langle H \rangle &= \partial_{p_k} \frac{\sum_{\mathbf{x}, \mathbf{x}'} \Psi^*(\mathbf{x}) H_{\mathbf{x}\mathbf{x}'} \Psi(\mathbf{x}')}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \\ &= \frac{\sum_{\mathbf{x}, \mathbf{x}'} \Psi^*(\mathbf{x}) H_{\mathbf{x}\mathbf{x}'} D_k(\mathbf{x}') \Psi(\mathbf{x}')}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} + \frac{\sum_{\mathbf{x}, \mathbf{x}'} \Psi^*(\mathbf{x}) D_k^*(\mathbf{x}) H_{\mathbf{x}\mathbf{x}'} \Psi(\mathbf{x}')}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \\ &\quad - \frac{\sum_{\mathbf{x}, \mathbf{x}'} \Psi^*(\mathbf{x}) H_{\mathbf{x}\mathbf{x}'} \Psi(\mathbf{x}')}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \frac{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2 (D_k(\mathbf{x}) + D_k^*(\mathbf{x}))}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \\ &= \frac{\sum_{\mathbf{x}, \mathbf{x}'} \frac{\Psi^*(\mathbf{x})}{\Psi^*(\mathbf{x}')} H_{\mathbf{x}\mathbf{x}'} D_k(\mathbf{x}') |\Psi(\mathbf{x}')|^2 + \sum_{\mathbf{x}, \mathbf{x}'} |\Psi(\mathbf{x})|^2 H_{\mathbf{x}\mathbf{x}'} D_k^*(\mathbf{x}') \frac{\Psi(\mathbf{x}')}{\Psi(\mathbf{x})}}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \\ &\quad - \langle H \rangle \frac{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2 (D_k(\mathbf{x}) + D_k^*(\mathbf{x}))}{\sum_{\mathbf{x}} |\Psi(\mathbf{x})|^2} \\ &\simeq \langle \langle E_{\text{loc}} D_k^* \rangle \rangle - \langle \langle E_{\text{loc}} \rangle \rangle \langle \langle D_k^* \rangle \rangle + cc. \end{aligned} \quad (3.14)$$

We can therefore compactly write $\partial_{p_k} \langle H \rangle \simeq \langle \langle G_k \rangle \rangle$, with the gradient estimator being

$$G_k(\mathbf{x}) = 2\text{Re}[(E_{\text{loc}}(\mathbf{x}) - \langle \langle E_{\text{loc}} \rangle \rangle) D_k^*(\mathbf{x})]. \quad (3.15)$$

3.2.1 Zero-Variance Property

One of the most interesting feature of the energy and energy-gradient estimators so far presented is that they have the so-called zero-variance property: their statistical fluctuations are exactly zero when sampling from the exact ground-state wave-function. Let us consider for example

$$\begin{aligned} \text{var}(E_{\text{loc}}) &= \langle E_{\text{loc}}^2 \rangle - \langle E_{\text{loc}} \rangle^2 \\ &= \sum_{\mathbf{x}} \Psi(\mathbf{x})^2 E_{\text{loc}}(\mathbf{x})^2 - \langle H \rangle^2 \\ &= \sum_{\mathbf{x}} \sum_{\mathbf{x}_1} H_{\mathbf{x}, \mathbf{x}_1} \Psi(\mathbf{x}_1) \sum_{\mathbf{x}_2} H_{\mathbf{x}, \mathbf{x}_2} \Psi(\mathbf{x}_2) - \langle H \rangle^2 \\ &= \sum_{\mathbf{x}_1} \Psi(\mathbf{x}_1) \sum_{\mathbf{x}} H_{\mathbf{x}, \mathbf{x}_1} \sum_{\mathbf{x}_2} H_{\mathbf{x}, \mathbf{x}_2} \Psi(\mathbf{x}_2) - \langle H \rangle^2 \\ &= \langle H^2 \rangle - \langle H \rangle^2, \end{aligned} \quad (3.16)$$

where we have assumed for simplicity that the wave-function is real. Therefore the variance of the local energy is an important physical quantity: the energy variance. It is easy to see that if Ψ is an eigenstate of H then $\langle H^2 \rangle = \langle H \rangle^2 = E_0^2$, and $\text{var}(E_{\text{loc}}) = 0$, i.e. the statistical fluctuations completely vanish. This property is very important since it also implies that, in a sense to be specified below, the closer we get to the ground-state, the less fluctuations we have on the quantity we want to minimize, the energy.

3.2.2 Stochastic Gradient Descent

Like all the other learning paradigms we have seen before, also in this case we can use SGD to optimize the parameters of the network. In particular the SGD iterate is:

$$p_k^{s+1} = p_k^s - \eta \partial_{p_k} \langle H \rangle. \quad (3.17)$$

An important difference with respect to the non-stochastic (deterministic) gradient descent approach, is that now we only have stochastic averages of the gradient which is therefore subjected to noise. Like in the previous discussion, also in this case we can define an effective Langevin temperature, which reads:

$$T = \frac{\eta \text{var}(\bar{G}_k)}{2} \quad (3.18)$$

$$\propto \frac{\eta}{N_s} \text{var}(G_k). \quad (3.19)$$

An important difference that occurs during the optimization is that typically it is not necessary to reduce the learning rate when going through the iterations. The main reason is that when we start approaching the true ground state, then $\text{var}(G_k) \rightarrow 0$. Indeed, also the gradient enjoys a zero-variance principle very similar to the one previously discussed for the local energy estimator. In particular it is easy to show that for an exact eigenstate the statistical fluctuations of the gradient are exactly vanishing, i.e. $\text{var}(G_k) = 0$. In practice then, even a constant number of samples and a fixed (small) η are sufficient to converge to the ground-state, provided that one checks during the optimization that the value of the effective temperature (3.18) is actually going to zero as expected.

3.3 Example: Jastrow Factors

Let us give a specific example of variational states, we consider now a system of interacting particles in continuous space, for which the most general Hamiltonian is

$$H = -\frac{\hbar^2}{2m} \sum_i^N \nabla_{\vec{r}_i}^2 + \sum_i V_1(\vec{r}_i) + \sum_{i<j} V_2(\vec{r}_i, \vec{r}_j), \quad (3.20)$$

where V_1 and V_2 are generic one and two-body interaction potential.

We now define the exact Jastrow-Feenberg expansion for the many-body state:

$$\begin{aligned} \Psi_p(\vec{r}_1, \dots, \vec{r}_N) = & \Psi_0(\vec{r}_1, \dots, \vec{r}_N) \times \exp \left[\sum_i J_1(\vec{r}_i) + \frac{1}{2} \sum_{i \neq j} J_2(\vec{r}_i, \vec{r}_j) + \right. \\ & \left. + \dots \frac{1}{p!} \sum_{i_1 \neq i_2 \neq \dots i_p} J_p(\vec{r}_{i_1}, \vec{r}_{i_2} \dots \vec{r}_{i_p}) \right], \end{aligned} \quad (3.21)$$

where $\Psi_0(\vec{r}_1, \dots, \vec{r}_N)$ is some parameter-independent wave-function, and the variational parameters are the functions $J_1(\vec{r}), J_2(\vec{r}, \vec{r}'), \dots, J_p(\vec{r}_1, \vec{r}_2 \dots \vec{r}_p)$. These expansion is clearly exact when $p = N$, however in practice one observes convergence to the exact ground-state much sooner, and typically $p = 2, 3$ are enough to obtain very accurate results.

3.3.1 One-dimensional trapped particles

As a simple exercise one can consider single-particle, one-dimensional Hamiltonians of trapped particles, for which $V_1(x)$ is an even function of x and $V_2 = 0$ (non-interacting particles). In this case (for symmetry reasons) one can write the function expansion $J_1(x) = p_1 x^2 + p_2 x^4 + \dots$, where p_1, p_2 etc are the parameters to be determined in a variational way. With this choice, one can easily derive expressions for the local energy, using the fact that:

$$\frac{1}{\Psi(x)} \frac{\partial^2}{\partial x^2} \Psi(x) = J_1''(x) + J_1'(x)^2. \quad (3.22)$$

In this case it is easy to show that

$$E_{\text{loc}}(x) = -\frac{\hbar^2}{2m} (J_1''(x) + J_1'(x)^2) + V_1(x) \quad (3.23)$$

$$D_k(x) = x^{2k}. \quad (3.24)$$

3.4 Neural-Network quantum states

In the first part of this Lecture we have rephrased the problem of finding a ground state in terms of a stochastic optimization problem. To really take advantage of the potentialities of machine learning, however it is still necessary to accomplish task 1, in our previous list: we need to define a suitable machine to solve our learning problem. This is what we have done in our recent work. [5]

The fundamental problem with the stochastic optimization problem described before is that, in principle, to achieve the exact ground state energy one needs to consider exponentially many parameters. To see this point, consider the case of N spin 1/2 particles, then the exact ground-state wave-function is fully specified by the 2^N amplitudes

$$\langle \mathbf{x} | \Psi \rangle = \Psi(\mathbf{x}), \quad (3.25)$$

for all the possible values of $\mathbf{x} = \sigma_1^z \sigma_2^z \dots \sigma_N^z$.

This task however is clearly unfeasible when the number of particles N is too large. For example, one can do a back-of-the-envelope calculation to show that only *storing* the wave-function for more than 100 spins would require a number of atoms larger than what can be found on our planet!

However, this exponential complexity is not necessary a limiting factor. In this case we can indeed think of using the ability of artificial neural networks to compress high-dimensional data into a low-dimensional representation.

The starting point is to ask a suitable neural network to *compute* the wave-function amplitudes. Formally, we then set:

$$\Psi(\mathbf{x}) = F(\mathbf{x}; p_1, p_2 \dots p_{N_p}), \quad (3.26)$$

where F is the output of a suitably chosen artificial neural network, depending on a set of parameters $\mathbf{p}$.

3.5 Using Restricted Boltzmann Machines

During the lecture I will show an example implementation of the stochastic optimization algorithm for neural-network RBM states. In particular, I will consider the transverse-field Ising hamiltonian in 1D:

$$H = -h \sum_i \sigma_i^x - J \sum_i \sigma_i^z \sigma_{i+1}^z, \quad (3.27)$$

with periodic boundary conditions over a ring of L sites. To simplify things, and knowing that the ground-state wave-function in this case is positive definite, I will consider the following quantum state [6]:

$$\Psi(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z) = \sqrt{F_{\text{rbm}}(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z)}, \quad (3.28)$$

where the specific RBM taken here contains only real-valued parameters. An advantage of this formulation is that sampling from $|\Psi(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z)|^2 = F_{\text{rbm}}(\sigma_1^z, \sigma_2^z, \dots, \sigma_N^z)$ is particularly easy, since it can be done using alternate Gibbs sampling, as previously discussed in the context of unsupervised learning.

3.5.1 Computing the local energy

For a given spin configuration, we also need to compute the local energy:

$$E_{\text{loc}}(\sigma) = \sum_{\sigma'} H_{\sigma, \sigma'} \frac{\psi(\sigma')}{\psi(\sigma)}. \quad (3.29)$$

For the transverse-field Ising model, the sum runs over the $N+1$ configurations $\sigma'(0) = \sigma$ and $\sigma'(k) = \sigma_1^z \dots \sigma_k^z \dots \sigma_N^z$, with $H_{\sigma, \sigma} = -J \sum_i \sigma_i^z \sigma_{i+1}^z$ and $H_{\sigma, \sigma'(k>0)} = -h$. This sum can then be computed in polynomial time, and it is efficiently done pre-computing the values of the “angles” θ_j . In particular,

$$\frac{\psi(\sigma'(k))}{\psi(\sigma)} = e^{-2a_k \sigma_k} \times \prod_j \frac{\cosh(\theta_j - 2\sigma_k W_{jk})}{\cosh(\theta_j)}. \quad (3.30)$$

3.5.2 Computing the variational derivatives

The variational derivatives

$$D_k(\sigma) = \frac{\partial_{p_k} \Psi(\sigma)}{\Psi(\sigma)}, \quad (3.31)$$

can also be computed efficiently and read:

$$D_{a_i}(\sigma) = \frac{1}{2} \sigma_i^z, \quad (3.32)$$

$$D_{b_j}(\sigma) = \frac{1}{2} \tanh(\theta_j), \quad (3.33)$$

$$D_{W_{ij}}(\sigma) = \frac{1}{2} \tanh(\theta_j) \sigma_i^z. \quad (3.34)$$

Bibliography

- [1] D. A. Sprecher, “On the structure of continuous functions of several variables”, Transactions Amer. Math. Soc, 115, 340–355, 1965.
- [2] William L. McMillan, Phys. Rev. 138, A442 (1965)
- [3] Asja Fischer, and Christian Igel, “Training Restricted Boltzmann Machines: An Introduction”. *Pattern Recognition* 47, 25-39, 2014.
- [4] Geoffrey Hinton, “A practical guide to training restricted Boltzmann Machines”. <https://www.cs.toronto.edu/~hinton/absps/guideTR.pdf>
- [5] Giuseppe Carleo, and Matthias Troyer. “Solving the quantum many-body problem with artificial neural networks”. *Science* 355, 602-606, 2017.
- [6] Giacomo Torlai, Guglielmo Mazzola, Juan Carrasquilla, Matthias Troyer, Roger Melko, and Giuseppe Carleo. *arXiv* 1703.05334, 2017.

Appendix A

Sampling Methods

During the lecture we have established a fundamental connection between quantum mechanics and statistical sampling. For this mapping to be efficient, we need an efficient way of sampling from the probability distribution $\Pi(\mathbf{x}) = |\Psi(\mathbf{x})|^2$. In particular the goal is to generate N_s samples $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N_s)}$ such that we can estimate expectation values as averages over those samples:

$$\langle\langle O_{\text{loc}} \rangle\rangle \simeq \frac{1}{N_s} \sum_i O_{\text{loc}}(\mathbf{x}^{(i)}). \quad (\text{A.1})$$

A.0.1 Markov Chain and Detailed Balance

A Markov chain is completely specified by the transition probability $\mathcal{T}(\mathbf{x}^{(i)} \rightarrow \mathbf{x}^{(i+1)})$, i.e. given a sample $\mathbf{x}^{(i)}$, we transition to the next element of the chain with probability T . The transition probability (as all well-defined probabilities) must always be normalized: $\sum_{\mathbf{x}'} \mathcal{T}(\mathbf{x} \rightarrow \mathbf{x}') = 1$.

We would like to devise a Markov chain process such that $\Pi^{\text{mc}}(\mathbf{x}) = \Pi(\mathbf{x})$, i.e. that the probability with which a given state $\mathbf{x}$ appears in the chain is equal to desired probability we want to sample from.

An important condition for this to happen is that the probability distribution $\Pi^{\text{mc}}(\mathbf{x})$ is *stationary*, i.e. all states along the chain should be distributed according to the same probability, and this should not change along the chain. A sufficient condition for this to happen is that

$$\Pi(\mathbf{x})\mathcal{T}(\mathbf{x} \rightarrow \mathbf{x}') = \Pi(\mathbf{x}')\mathcal{T}(\mathbf{x}' \rightarrow \mathbf{x}), \quad (\text{A.2})$$

which is called *detailed balance* equation. This condition basically enforces stationarity (also called micro-reversibility) in the chain: the probability of being in a given state $\mathbf{x}$ and of doing a transition to another state $\mathbf{x}'$ must be equal to the reverse process, starting from $\mathbf{x}'$ and transitioning to $\mathbf{x}$.

A.0.2 The Metropolis-Hastings Algorithm

There exist many possible transition probabilities that satisfy the detailed balance condition (A.2), however the most famous choice is certainly the Metropolis-Hastings pre-

scription. In this case, we separate the transition process into two steps:

$$\mathcal{T}(\mathbf{x} \rightarrow \mathbf{x}') = T(\mathbf{x} \rightarrow \mathbf{x}')A(\mathbf{x} \rightarrow \mathbf{x}'), \quad (\text{A.3})$$

i.e. we first propose a state with some (simple) probability distribution $T(\mathbf{x} \rightarrow \mathbf{x}')$ we can easily sample from, and then accept or reject the new state $\mathbf{x}'$ as the next element of the chain with probability $A(\mathbf{x} \rightarrow \mathbf{x}')$.

Using the detailed balance condition, we see that the acceptance probability must satisfy:

$$\frac{A(\mathbf{x} \rightarrow \mathbf{x}')}{A(\mathbf{x}' \rightarrow \mathbf{x})} = \frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x})}{T(\mathbf{x} \rightarrow \mathbf{x}')} \quad (\text{A.4})$$

A possible acceptance that satisfies this condition is:

$$A(\mathbf{x} \rightarrow \mathbf{x}') = \min \left(1, \frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x})}{T(\mathbf{x} \rightarrow \mathbf{x}')} \right). \quad (\text{A.5})$$

Notice that this acceptance probability satisfies (A.4), since if $\frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x})}{T(\mathbf{x} \rightarrow \mathbf{x}')} < 1$ then $\frac{\Pi(\mathbf{x})}{\Pi(\mathbf{x}')} \times \frac{T(\mathbf{x} \rightarrow \mathbf{x}')}{T(\mathbf{x}' \rightarrow \mathbf{x})} > 1$, $A(\mathbf{x}' \rightarrow \mathbf{x}) = 1$ and (A.4) is trivially verified. The same reasoning can be applied for the case $\frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x})}{T(\mathbf{x} \rightarrow \mathbf{x}')} > 1$.

The Metropolis-Hasting Algorithm can be then summarized in the following steps:

1. Generate a random state $\mathbf{x}'$ drawing from the (simple) transition probability $T(\mathbf{x}^{(i)} \rightarrow \mathbf{x}')$.
2. Compute the quantity

$$R = \frac{\Pi(\mathbf{x}')}{\Pi(\mathbf{x}^{(i)})} \times \frac{T(\mathbf{x}' \rightarrow \mathbf{x}^{(i)})}{T(\mathbf{x}^{(i)} \rightarrow \mathbf{x}')} \quad (\text{A.6})$$

3. Draw a uniformly distributed random number $\eta \in [0, 1)$.
4. If $R > \eta$, accept the new states, i.e. $\mathbf{x}^{(i+1)} = \mathbf{x}'$. Otherwise, the following state in the chain stays the current one: $\mathbf{x}^{(i+1)} = \mathbf{x}^{(i)}$.

Notice that steps 2-4 are necessary to decide whether to accept or reject the proposed state according to the Metropolis probability (A.5).

Appendix B

Estimating Errors and Auto-Correlation Times

Since Markov chains are generated transitioning from a state to the next one, it is natural to expect that adjacent points in the chain will be statistically correlated. To quantify this notion of correlation more precisely, let us first consider the Markov chain estimate for the expectation value of a given function:

$$\bar{g}_{n_s} = \frac{1}{n_s} \sum_i^{n_s} g_i, \quad (\text{B.1})$$

where we have used the short-hand $g_i \equiv g(\mathbf{x}^{(i)})$. The law of large numbers states that

$$\bar{g}_{n_s} \xrightarrow[n_s \rightarrow \infty]{} \sum_{\mathbf{x}} \Pi(\mathbf{x}) g(\mathbf{x}), \quad (\text{B.2})$$

and the central limit theorem says that $\bar{g}_{n_s}$ is a random variable normally distributed,

$$\text{Prob}(\bar{g}_{n_s}) = \text{Normal}(\bar{g}_{\infty}, \sigma^2), \quad (\text{B.3})$$

with expected value $\bar{g}_{\infty}$ and variance $\sigma^2 = \text{var}(\bar{g}_{n_s})$, where the variance is computed over different realizations of the Markov chain. It explicitly reads

$$\begin{aligned} \text{var}(\bar{g}_{n_s}) &= \text{var} \left(\frac{1}{n_s} \sum_i g_i \right) \\ &= \frac{1}{n_s^2} \sum_{ij} \langle g_i g_j \rangle - \frac{1}{n_s^2} \sum_{ij} \langle g_i \rangle \langle g_j \rangle \\ &= \frac{1}{n_s} \left(\frac{1}{n_s} \sum_i (\langle g_i^2 \rangle - \langle g_i \rangle^2) + \frac{2}{n_s} \sum_i \sum_{j=i+1} (\langle g_i g_j \rangle - \langle g_i \rangle \langle g_j \rangle) \right) \\ &= \frac{1}{n_s} \text{var}(g_0) + 2 \sum_{j=1}^{n_s} (\langle g_0 g_j \rangle - \langle g_0 \rangle \langle g_j \rangle) \left(1 - \frac{j}{n_s} \right), \end{aligned} \quad (\text{B.4})$$

where we assumed that the Markov chain is stationary, i.e. $\text{var}(g_i)$ does not depend on the index i and the same for the covariance. Therefore

$$\text{var}(\bar{g}_{n_s}) = \frac{1}{n_s} \text{var}(g_0) 2\tau_{\text{int}}, \quad (\text{B.5})$$

having defined the integrated auto-correlation time as

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{j=1}^{n_s} (\langle g_0 g_j \rangle - \langle g_0 \rangle \langle g_j \rangle) \left(1 - \frac{j}{n_s}\right). \quad (\text{B.6})$$

We therefore see that unless the Markov chain samples are completely uncorrelated (i.e. $\langle g_s g_j \rangle - \langle g_s \rangle \langle g_j \rangle = 0$) the statistical error on the estimator $\bar{g}_{n_s}$ is increased by the positive factor τ_{int} .

A way to correctly estimate the integrated autocorrelation time is through the correlation function

$$\rho(j) = \frac{\langle g_0 g_j \rangle - \langle g \rangle^2}{\langle g^2 \rangle - \langle g \rangle^2}, \quad (\text{B.7})$$

and a numerically stable estimate of the correlation time is given by

$$\tau_{\text{int}} \simeq \frac{1}{2} + \sum_{j=1}^{j_{\text{cut}}} \rho(j), \quad (\text{B.8})$$

where j_{cut} is chosen for numerical stability as the first j such that $\rho(j_{\text{max}}) < 0$. In practice, given a sequence of estimates $g_1, \dots, g_{n_s} = \mathbf{g}$, then the correlation function can be efficiently estimated with a sequence of Fast Fourier Transforms and its inverses:

$$A = FFT(\mathbf{g} - \bar{g}), \quad (\text{B.9})$$

$$B = AA^*, \quad (\text{B.10})$$

$$\rho = \frac{FFT^{-1}(B)}{\langle g^2 \rangle - \langle g \rangle^2}. \quad (\text{B.11})$$